3_Práctica NFR24L01 - Envío de datos con generador de señal

Práctica: Enlace inalámbrico con módulos NRF24L01 (TX y RX) con el uso de un sensor.

Fecha: 16/03/2026

Integrantes:
-Eduardo Cadengo López
-Itzel Citlalli Martell De La Cruz
-Damian Alexander Díaz Piña

Contenido

Descripción de la ESP32 1
Descripción del NRF24L01 1
Diferenciación: NRF24L01 con/sin PA+LNA y con/sin antena 2
Descripción del funcionamiento del sensor ultrasónico (HC-SR04) 2
Diagrama de conexión 2
TX: Arduino UNO + HC-SR04 + NRF24L01 2
RX: ESP32 + NRF24L01 2
Código comentado 3
Conclusiones personales 5
URL a GitHub 5
URL a evidencia en video 5

Descripción de la ESP32

La ESP32 es un SoC con CPU dual Tensilica LX6 hasta ~240 MHz, conectividad Wi-Fi 2.4 GHz y Bluetooth BLE, y una amplia cantidad de periféricos: SPI, I²C, UART, PWM, ADC/DAC, etc. En esta práctica se usa como receptor (RX) controlando el módulo NRF24L01 por SPI y mostrando los datos en el monitor serie.

Descripción del NRF24L01

El NRF24L01+ es un transceptor en la banda ISM de 2.4 GHz con interfaz SPI. Permite hasta 6 pipes de recepción con direcciones, payloads de hasta 32 bytes, auto-ACK y reintentos

automáticos. Opera en 125 canales (0–124) y ofrece tasas de 250 kbps, 1 Mbps y 2 Mbps. Se alimenta a 3.3 V y requiere buen desacoplo cercano al módulo.

Diferenciación: NRF24L01 con/sin PA+LNA y con/sin antena

• NRF24L01 estándar (antena en PCB): consumo menor (~12–15 mA RX/TX), alcance corto-medio, ideal para prácticas de mesa.
• NRF24L01+ PA+LNA (conector SMA y antena externa): mayor potencia de transmisión y mejor sensibilidad, requiere picos >100 mA en TX y una fuente 3.3 V estable; alcance medio-largo.

Descripción del funcionamiento del sensor ultrasónico (HC-SR04)

El HC-SR04 mide distancia emitiendo un pulso ultrasónico de 40 kHz al activar TRIG por 10 µs. El pin ECHO permanece en alto durante el tiempo de vuelo (ida y vuelta) de la onda; midiendo ese pulso con pulseIn() se calcula la distancia como (duración × 0.034) / 2, en centímetros.

Diagrama de conexión

TX: Arduino UNO + HC-SR04 + NRF24L01

Elemento/Señal	Arduino UNO
NRF24L01 – VCC	3.3 V (NO 5 V)
NRF24L01 – GND	GND
NRF24L01 – CE	D9
NRF24L01 – CSN	D10
NRF24L01 – SCK	D13
NRF24L01 – MOSI	D11
NRF24L01 – MISO	D12
HC-SR04 – TRIG	D6
HC-SR04 – ECHO	D7

RX: ESP32 + NRF24L01

NRF24L01 – Señal	ESP32 (VSPI)
VCC	3.3 V
GND	GND
CE	GPIO 4
CSN (SS)	GPIO 5
SCK	GPIO 18
MOSI	GPIO 23
MISO	GPIO 19

Código comentado

```
// --- Arduino UNO (TX) ---

// Lee distancia del HC-SR04 y la envía como float por NRF24L01
#include <RF24.h>

RF24 radio(9,10); // CE, CSN
const byte direccion[6] = "00001"; // Pipe de escritura

#define TRIG 6
#define ECHO 7

long duracion;     // tiempo en microsegundos del pulso ECHO
float distancia;   // distancia en cm

void setup() {
  Serial.begin(9600);

  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);

  if (!radio.begin()) {
    Serial.println("NRF24L01 no detectado");
    while(1); // detente si no hay radio
  }

  radio.openWritingPipe(direccion);   // establece destino
  radio.setChannel(112);               // canal 112 (debe coincidir con
RX)
  radio.setPALevel(RF24_PA_LOW);       // potencia baja para pruebas de
mesa
  radio.stopListening();               // modo transmisión

  Serial.println("Transmisor listo con sensor ultrasonico");
}

void loop() {
  // Disparo ultrasónico
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);

  duracion = pulseIn(ECHO, HIGH);           // mide ancho del pulso
```

```
  distancia = duracion * 0.034 / 2.0;       // convierte a cm (ida y
vuelta)

  bool ok = radio.write(&distancia, sizeof(distancia));
  if (ok) {
    Serial.print("Distancia enviada: ");
    Serial.print(distancia); Serial.println(" cm");
  } else {
    Serial.println("Error al enviar");
  }

  delay(1000); // periodo ~1 s
}
```

```
// --- ESP32 (RX) ---

// Recibe distancia (float) por NRF24L01 y la muestra por Serial
#include <SPI.h>
#include <RF24.h>

RF24 radio(4,5); // CE, CSN
const byte direccion[6] = "00001"; // Debe igualar al TX
float distanciaRecibida;

void setup() {
  Serial.begin(115200);

  // (Opcional) asegura pines de VSPI explícitos en algunas placas
  // SPI.begin(18, 19, 23, 5); // SCK, MISO, MOSI, SS

  if (!radio.begin()) {
    Serial.println("NRF24L01 no detectado");
    while(1);
  }

  radio.openReadingPipe(0, direccion); // abre pipe 0
  radio.setChannel(112);               // mismo canal que el TX
  radio.setPALevel(RF24_PA_LOW);
```

```
  radio.startListening();              // modo recepción

  Serial.println("Receptor listo, esperando datos...");
}

void loop() {
  if (radio.available()) {
    radio.read(&distanciaRecibida, sizeof(distanciaRecibida));
    Serial.print("Distancia recibida: ");
    Serial.print(distanciaRecibida);
    Serial.println(" cm");
  }
}
```

Conclusiones personales

- Eduardo:

- Itzel:

- Damian:

URL a GitHub

https://github.com/AresGodKiller/Tecnologias_Inalambricas/tree/main/3_Practica%20NFR24L01_Sensor

URL a evidencia en video

https://youtu.be/xSb_0cPzFfY